

On graph problems in a semi-streaming model[☆]

Joan Feigenbaum^{a,1,2}, Sampath Kannan^{b,2,3}, Andrew McGregor^{b,*,2,4},
Siddharth Suri^{b,2,5}, Jian Zhang^{a,2,6}

^aDepartment of Computer Science, Yale University, New Haven, CT 06520, USA

^bDepartment of Computer and Information Science, University of Pennsylvania, Philadelphia, PA 19104, USA

Abstract

We formalize a potentially rich new streaming model, the *semi-streaming model*, that we believe is necessary for the fruitful study of efficient algorithms for solving problems on massive graphs whose edge sets cannot be stored in memory. In this model, the input graph, $G = (V, E)$, is presented as a stream of edges (in adversarial order), and the storage space of an algorithm is bounded by $O(n \cdot \text{polylog } n)$, where $n = |V|$. We are particularly interested in algorithms that use only one pass over the input, but, for problems where this is provably insufficient, we also look at algorithms using constant or, in some cases, logarithmically many passes. In the course of this general study, we give semi-streaming constant approximation algorithms for the unweighted and weighted matching problems, along with a further algorithmic improvement for the bipartite case. We also exhibit $\log n / \log \log n$ semi-streaming approximations to the diameter and the problem of computing the distance between specified vertices in a weighted graph. These are complemented by $\Omega(\log^{(1-\varepsilon)} n)$ lower bounds.

© 2005 Published by Elsevier B.V.

Keywords: Graph; Streaming; Matching; Spanner; Girth; Articulation point

1. Introduction

Streaming [14,2,10] is an important model for computation on massive data sets. Recently, there has been a large body of work on designing algorithms in this model [11,2,10,15,13,12]. Yet, the problems considered fall into a small number of categories, such as computing statistics, norms, and histograms. Very few graph problems [4] have been considered in the streaming model.

[☆] This work was supported by the DoD University Research Initiative (URI) administered by the Office of Naval Research under Grant N00014-01-1-0795.

* Corresponding author.

E-mail addresses: feigenbaum-joan@cs.yale.edu (J. Feigenbaum), kannan@cis.upenn.edu (S. Kannan), andrewm@cis.upenn.edu (A. McGregor), ssuri@gradient.cis.upenn.edu (S. Suri), zhang-jian@cs.yale.edu (J. Zhang).

¹ Supported in part by ONR Grant N00014-01-1-0795 and NSF Grant ITR-0331548.

² Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

³ Supported in part by ARO Grant DAAD 19-01-1-0473 and NSF Grant CCR-0105337.

⁴ Supported in part by NSF grant ITR-0205456.

⁵ Supported in part by NIH Grant T32 HG000046-05.

⁶ Supported by ONR Grant N00014-01-1-0795 and NSF Grants CCR-0105337 and ITR-0331548.

The difficulty of graph problems in the streaming model arises from the memory limitation of the model combined with input-access constraints. We can view the amount of memory used by algorithms with sequential (one-way) input access as a spectrum. At one end of the spectrum, we have dynamic algorithms [9] that may use memory enough for the whole input. At the other end, we have streaming algorithms that use only polylog space. At one extreme, there is a lot of work on dynamic graph problems; on the other, general graph problems are considered hard in the (poly)log-space streaming model. Recently, it has been suggested by Muthukrishnan [19] that the middle ground, where the algorithms can use $O(n \cdot \text{polylog } n)$ bits of space, is an interesting and open area. This is the area that we explore.

Besides taking a middle position in the memory-size spectrum, the semi-streaming model allows multiple passes over the input stream. In certain applications with massive data sets, a small number of sequential passes over the data would be much more efficient than many random accesses to the data. Only a few works [8,6] have considered the multiple-pass model and a lot remains to be done.

Massive graphs arise naturally in many real-world scenarios. Two examples are the *call graph*, where nodes correspond to telephone numbers and edges to calls between numbers that call each other during some time interval, and the *web graph*, where nodes are web pages, and the edges are links between pages. The streaming model is necessary for the study of the efficient processing of such massive graphs. In [1], the authors introduce the semi-external model for computations on massive graphs, i.e., one in which the vertex set can be stored in memory, but the edge set cannot. However, this work addresses the problems in an external memory model in which random access to the edges, although expensive, is allowed. This is a major difference between their model and ours. Indeed, the authors of [18] argue that one of the major drawbacks of standard graph algorithms, when applied to massive graphs such as the web, is their need to have random access to the edge set. Furthermore, there are situations in which the graph is revealed in a streaming fashion, such as a web crawler exploring the web graph.

We consider a set of classical graph problems in this semi-streaming model. We show that, although the computing power of this model is still limited, there are semi-streaming algorithms for a variety of graph problems. Our main result is a semi-streaming algorithm that computes a $(\frac{2}{3} - \epsilon)$ -approximation in $O((\log(1/\epsilon))/\epsilon)$ passes for unweighted bipartite graph matching. We also provide a one-pass semi-streaming algorithm for $\frac{1}{6}$ -approximating the maximum weighted graph matching. We also provide $\log n / \log \log n$ approximations for diameter and shortest paths in weighted graphs which we complement with $\Omega(\log^{(1-\epsilon)} n)$ lower bounds for these problems in unweighted graphs.

2. Preliminaries

Unless stated otherwise, we denote by $G(V, E)$ a graph G with vertex set $V = \{v_1, v_2, \dots, v_n\}$ and edge set $E = \{e_1, e_2, \dots, e_m\}$. Note that n is the number of vertices and m the number of edges.

Definition 1. A *graph stream* is a sequence of edges $e_{i_1}, e_{i_2}, \dots, e_{i_m}$, where $e_{i_j} \in E$ and $i_1, i_2, \dots, i_m$ is an arbitrary permutation of $[m] = \{1, 2, \dots, m\}$.

While an algorithm goes through the stream, the graph is revealed one edge at a time. This definition generalizes the streams of graphs in which the adjacency matrix or the adjacency list is presented as a stream. In a stream in the adjacency-matrix or adjacency-list models, the edges incident to each vertex are grouped together. We need the more general model to account for graphs such as call graphs where the edges might generated in any order.

The efficiency of a graph algorithm in the semi-streaming model is measured by the space it uses, the time it requires to process each edge, and the number of passes it makes over the graph stream.

Definition 2. A *semi-streaming graph algorithm* computes over a graph stream using $S(n, m)$ bits of space. The algorithm may access the input stream in a sequential order(one-way) for $P(n, m)$ passes and use $T(n, m)$ time to process each edge. It is required that $S(n, m)$ be $O(n \cdot \text{polylog } (n))$ bits.

To see the limitation of the (poly)log-space streaming model for graph problems, consider the following simple problem. Given a graph, determining whether there is a length-2 path between two vertices, x and y , is equivalent to deciding whether two vertex sets, the neighborhood of x and the neighborhood of y have a non-empty intersection. Because set disjointness has linear-space communication complexity [16], the length-2 path problem is impossible in

the (poly)log-space streaming model. See [6] for a more comprehensive treatment of finding common neighborhoods in the streaming model.

3. Graph matching

3.1. Unweighted bipartite matching

In this subsection, we present an algorithm for approximating unweighted bipartite matching. First, a bipartition can be found by the following algorithm. Note that the labeling of the vertices keeps track of the connected components in the graph seen so far, and the signs keep a partition for each connected component.

Algorithm 1 (*Bipartition*). *As edges stream in, we use a disjoint set data structure to maintain the connected components of the graph so far. We associate a sign with each vertex such that no edge joins two vertices of the same sign. If this condition ever fails and cannot be corrected by flipping the sign of a vertex and the vertices in its connected component, then we output that the graph is non-bipartite.*

The disjoint set data structure with union by rank and path compression can be augmented to maintain the signs without increasing the amortized time, $\alpha(m, n)$ needed per edge.

Given a matching M , we call a vertex *free* if it does not appear as the end point of any edge in M . It is easy to see that a maximal matching (thus a $\frac{1}{2}$ -approximation to maximum matching) for a graph can be constructed by a semi-streaming algorithm in one pass over the graph stream: when going through the stream, the algorithm adds an edge to the current matching M if both ends of the edge are free w.r.t. M . (This constructs a maximal matching for an arbitrary graph, not just for bipartite graphs.)

Consider a matching M for a bipartite graph $G = (L \cup R, E)$. A length-3 augmenting path for an edge $e = (u, v) \in M$, $u \in L$ and $v \in R$ is a quadruple (w_l, u, v, w_r) such that $(u, w_l), (w_r, v) \in E$, and w_l and w_r are free vertices. We call w_l and w_r the *wing-tips* of the augmenting path, (u, w_l) the *left wing* and (w_r, v) the *right wing*. A set of *simultaneously augmentable length-3 augmenting paths* is a set of length-3 augmenting paths that are vertex disjoint.

We now provide an algorithm that will be used as a subroutine in our main unweighted bipartite matching algorithm. Given a bipartite graph and a matching of the graph, this algorithm finds a set of simultaneously augmentable length-3 augmenting paths.

Algorithm 2 (*Find augmenting paths*). *The input to the algorithm is a graph $G = (L \cup R, E)$, a matching M for G and a parameter $0 < \delta < 1$.*

- (1) *In one pass, find a maximal set of disjoint left wings. If the number of left wings found is $\leq \delta M$, terminate.*
- (2) *In a second pass, for the edges in M with left wings, find a maximal set of disjoint right wings.*
- (3) *In a third pass we identify the set of vertices that*
 - (a) *Are endpoints of a matched edge that got a left wing.*
 - (b) *Are the wing tips of a matched edge that got both wings.*
 - (c) *Are endpoints of a matched edge that is no longer 3 augmentable.*

We remember these vertices and in subsequent passes, we ignore any edge incident on one of these vertices.

- (4) *Repeat.*

Our main unweighted bipartite matching algorithm increases the size of a matching by repeatedly finding a set of simultaneously augmentable length-3 augmenting paths and augmenting the matching using these paths.

Algorithm 3 (*Unweighted bipartite matching*). *The input to the algorithm is a bipartite graph $G = (L \cup R, E)$ and a parameter $0 < \varepsilon < \frac{1}{3}$.*

- (1) *In one pass, find a maximal matching M and the bipartition of G .*
- (2) *For $k = 1, 2, \dots, \lceil (\log(6\varepsilon))/(\log(8/9)) \rceil$ Do:*
 - (a) *Run the Algorithm 2 with G, M and $\delta = \varepsilon/(2 - 3\varepsilon)$.*
 - (b) *For each $e = (u, v) \in M$ for which an augmenting path (w_l, u, v, w_r) is found by Algorithm 2, remove (u, v) from M and add (u, w_l) and (w_r, v) to M .*

We now establish a relationship between the size of a maximal set of simultaneously augmentable length-3 augmenting paths and the size of a maximum such set.

Lemma 1. *The size of a maximal set of simultaneously augmentable length-3 augmenting paths is at least $\frac{1}{3}$ of the size of a maximum set of simultaneously augmentable length-3 augmenting paths.*

Proof. Let $AP_{\max}$ be some maximum set of simultaneously length-3 augmentable paths. Note that each path in the maximal set destroys at most 3 paths that $AP_{\max}$ might have used, one involving each of the wing tips used and a third path involving the matched edge used. Thus, we have a $\frac{1}{3}$ -approximation. $\square$

The following lemma and its proof can be found, for example, [17, p. 156]. We present them here for completeness.

Lemma 2. *Let X be a maximum-sized set of simultaneously augmentable length-3 augmenting paths for a maximal matching M . Let $\alpha = |X|/|M|$ and OPT a maximum matching. $|M|(1 + \alpha) \geq \frac{2}{3}|OPT|$.*

Proof. Consider a maximum matching OPT and the symmetric difference $OPT \nabla M$. In each connected component, there is at most 1 more edge from OPT than there is from M . Note that no connected component consists of only a single edge that came from OPT , because M is maximal. The number of components with one edge from M and two edges from OPT is at most $|X|$. In all other components, the ratio of edges from M to edges from OPT is at least 2 : 3. The result follows. $\square$

Lemma 3. *Algorithm 2 finds $(\alpha|M| - 2\delta|M|)/3$ simultaneously augmentable length-3 augmenting paths in $3/\delta$ passes.*

Proof. Let $L(M)$ be the set of the end vertices of the edges in M that are in L and $V_L(M) = \{v \in R | v \text{ is free w.r.t. } M \text{ and } \exists u \in L(M) \text{ s.t. } (u, v) \in E\}$. We call one repetition of step 1–4 in the algorithm a phase. The number of phases is at most $1/\delta$ because at least $\delta|M|$ edges in M are removed at each phase.

When the algorithm terminates, the number of left wings found is at most $\delta|M|$. Note that the set of left wings found form a maximal matching between the remaining vertices in $L(M)$ and $V_L(M)$. Hence there are fewer than $2\delta|M|$ disjoint left wings that could have been found at this phase. Consequently, there are fewer than $2\delta|M|$ simultaneously augmentable length-3 augmenting paths in the remaining graph, which we denote G' .

Let $G'' = G \setminus G'$. Note that a maximum set of simultaneously augmentable length-3 augmenting paths in G'' would have a size at least $\alpha|M| - 2\delta|M|$. Also note that the set of length-3 augmenting paths found by the algorithm form a maximal set w.r.t. G'' . By Lemma 1, the size of such a set is at least $(\alpha|M| - 2\delta|M|)/3$. $\square$

Theorem 1. *For any $0 < \varepsilon < \frac{1}{3}$ and a bipartite graph, Algorithm 3 finds a $\frac{2}{3} - \varepsilon$ approximation of maximum matching in $O(\log 1/\varepsilon)/\varepsilon$ passes. The algorithm processes each edge in $O(1)$ time in each pass except the first pass, in which the bipartition is found. The amortized per-edge processing time is $\alpha(m, n)$ for finding the bipartition. The storage space required by the algorithm is $O(n \log n)$.*

Proof. It is easy to see the bounds for the per-edge processing time and storage space. We now show the correctness of the algorithm. Let OPT be the size of the maximum matching. At the i th phase, let M_i be the matching found by the algorithm and X_i a maximum-sized set of simultaneously augmentable length-3 augmenting paths for the matching M_i . Let $\alpha_i = |X_i|/|M_i|$ and $s_i = |M_i|/OPT$.

Note that we only need to consider the case where $\alpha_i > 3\varepsilon/(2 - 3\varepsilon)$. Otherwise, by Lemma 2, the matching M_i is already a $\frac{2}{3} \cdot 1/(1 + \alpha_i) \geq \frac{2}{3} - \varepsilon$ approximation. Assuming $\alpha_i > 3\varepsilon/(2 - 3\varepsilon)$ for all stage i , let $\delta = \varepsilon/(2 - 3\varepsilon)$. Then $\delta \leq \alpha_i/3$ for all α_i . By Lemma 3, the number of simultaneously augmentable length-3 augmenting paths found by Algorithm 2 is then $(\alpha_i|M_i| - 2\delta|M_i|)/3 \geq \alpha_i|M_i|/9$.

Because M_0 is a maximal matching, $s_0 \geq \frac{1}{2}$. At any stage, by Lemma 2, $|M_i| + \alpha_i|M_i| \geq \frac{2}{3} \cdot OPT$. This gives

$$s_i + \alpha_i s_i \geq 2/3. \quad (1)$$

By Lemma 3, $|M_{i+1}| = |M_i| \cdot (1 + (\alpha_i - 2\delta)/3) \geq |M_i| \cdot (1 + \alpha_i/9)$. This gives

$$s_{i+1} \geq s_i + \alpha_i s_i / 9. \quad (2)$$

Putting together inequalities 1 and 2, we have: $s_{i+1} \geq \frac{8}{9} \cdot s_i + \frac{2}{27}$. Solving this recurrence gives $s_i \geq \frac{2}{3} - \frac{1}{6}(\frac{8}{9})^i$. Note that the algorithm runs in $k = \lceil \log 6\varepsilon / (\log 8/9) \rceil$ stages. Thus $|M_k|/\text{OPT} = s_k \geq \frac{2}{3} - \varepsilon$.

At each stage of Algorithm 3, we run Algorithm 2 as a subroutine. There are $k = \lceil \log 6\varepsilon / (\log 8/9) \rceil$ stages and each stage requires $(6 - 9\varepsilon)/\varepsilon$ passes. The total number of passes is then

$$\left\lceil \frac{\log 6\varepsilon}{\log 8/9} \right\rceil \frac{6 - 9\varepsilon}{\varepsilon} = O\left(\frac{\log 1/\varepsilon}{\varepsilon}\right). \quad \square$$

3.2. Weighted matching

In the weighted matching problem, every edge e has a weight $w(e)$. We seek the matching M for which $\sum_{e \in M} w(e)$ is maximized. This is also a well-studied problem when we do not restrict ourselves to the streaming model.

At least one existing algorithm [21] can easily be adapted to work in our model. For any $\varepsilon > 0$, the streaming version finds a weighted matching that is at least $1/(2 + \varepsilon)$ the optimal size using $O(\log_{1+\varepsilon/3} n)$ passes and $O(n \log n)$ storage. The algorithm works by geometrically grouping the weights into $\lceil \log_{1+\varepsilon/3} (\lceil 3/\varepsilon + 1.5 \rceil n) \rceil$ groups and then, for each group, starting at those with the largest weights, finding maximal matchings. Each maximal matching can be found with one pass. Further details and the proof of correctness can be found in [21].

We propose a new algorithm that uses only one pass to find a matching which is at least $\frac{1}{6}$ of the optimal size.

Algorithm 4 (Weighted matching). We maintain a matching M at all times. When we see a new edge e , we compare $w(e)$ with $2w(C)$, the sum of the weights of the edges of $C = \{e' \mid e' \in M \text{ and } e' \text{ and } e \text{ share an end point}\}$.

- If $w(e) > 2w(C)$, we update $M \leftarrow M \cup \{e\} \setminus C$.
- If $w(e) \leq 2w(C)$, we ignore e .

Theorem 2. In 1 pass and $O(n \log n)$ storage, we can construct a weighted matching that is at least $1/6$ of the optimal size.

Proof. For any set of edges S , let $w(S) = \sum_{e \in S} w(e)$. We say that an edge is *born* if it is ever part of M . We say that an edge is *killed* if it was born but subsequently removed from M by a newer, heavier edge. This new edge *murdered* the killed edge. We say an edge is a *survivor* if it is born and never killed. Let the set of survivors be S . The weight of the matching we find is therefore $w(S)$.

For each survivor e , let the *Trail of the Dead* leading to this edge be $T(e) = C_1 \cup C_2 \cup \dots$ where $C_0 = \{e\}$, $C_1 = \{\text{the edges murdered by } e\}$, and $C_i = \bigcup_{e' \in C_{i-1}} \{\text{the edges murdered by } e'\}$.

Claim. $w(T(e)) \leq w(e)$.

Proof of claim. For each murdering edge e , $w(e)$ is at least twice the cost of murdered edges, and an edge has at most one murderer. Hence, for all i , $w(C_i) \geq 2w(C_{i+1})$ therefore

$$2w(T(e)) = \sum_{i \geq 1} 2w(C_i) \leq \sum_{i \geq 0} w(C_i) = w(T(e)) + w(e).$$

The claim follows.

Now consider the optimal solution that includes edges $\text{OPT} = \{o_1, o_2, \dots\}$. We are going to charge the costs of edges in OPT to the survivors and their trails of the dead, $\bigcup_{e \in S} T(e) \cup \{e\}$. We hold an edge e in this set *accountable* to $o \in \text{OPT}$ if either $e = o$ or if o was not born because e was in M when o arrived. Note that, in the second case, it is possible for two edges to be accountable to o . If only one edge is accountable for o then we charge $w(o)$ to e . If two edges e_1 and e_2 are accountable for o , then we charge $w(o)w(e_1)/(w(e_1) + w(e_2))$ to e_1 and $w(o)w(e_2)/(w(e_1) + w(e_2))$ to e_2 . In either case, the amount charged by o to any edge e is at most $2w(e)$.

We now redistribute these charges as follows: (for distinct u_1, u_2, u_3) if $e = (u_1, v)$ gets charged by $o = (u_2, v)$, and e subsequently gets killed by $e' = (u_3, v)$, we transfer the charge from e to e' . Note that we maintain the property that the amount charged by o to any edge e is at most $2w(e)$ because $w(e') \geq w(e)$. What this redistribution of charges

achieves is that now every edge in a trail of the dead is only charged by one edge in OPT. Survivors can, however, be charged by two edges in OPT. We charge $w(\text{OPT})$ to the survivors and their trails of the dead, and hence

$$w(\text{OPT}_c) \leq \sum_{e \in S} 2w(T(e)) + 4w(e).$$

By Claim 1,

$$\sum_{e \in S} 2w(T(e)) + 4w(e) \leq 6w(S)$$

and the theorem follows. $\square$

3.3. Lower bounds

In contrast to the above results, it is worth noting that even to check whether an existing matching is maximum in 1 pass requires $\Omega(m)$ space. First, we prove that testing s, t connectivity in a directed graph requires $\Omega(m)$ space.

Lemma 4. *Testing for $s - t$ connectivity in a directed graph $G = (V, E)$ requires $\Omega(m)$ bits of space, where $|E| = m$.*

Proof. Consider the family $\mathcal{F}$ of graphs $G = (L \cup R \cup \{s, t\}, E)$, where the induced graph on $L \cup R$ is an arbitrary bipartite graph with $|L| = |R| = n$ and $m \leq n^2/2$ edges and all the edges are directed from L to R . Say the stream gives all the edges between L and R first, then one edge (s, l) and another (r, t) , where $l \in L$ and $r \in R$. At the point at which all the edges from L to R have appeared any correct algorithm must have a different memory configuration for each graph in $\mathcal{F}$ since there are continuations that will result in different answers for any two graphs in $\mathcal{F}$. Thus, the number of bits of space required is $\Omega(\log_2 |\mathcal{F}|)$ which is easily seen to be $\Omega(m)$. $\square$

Theorem 3. *Consider a bipartite graph $G = (L \cup R, E)$. Testing whether there exists an augmenting path from $s \in R$ to $t \in L$ requires $\Omega(m)$ bits of storage.*

Proof. Let the storage required be $S(n)$. Let $G = (\{v_1, \dots, v_n\}, E)$ be a directed graph and assume, without loss of generality, that $s = v_1$ and $t = v_n$. We will use the “test for an augmenting path” algorithm to answer the s, t directed-connectivity problem. We construct an undirected bipartite graph G' with nodes v_s, v_t such that there exists an augmenting path from v_s to v_t in G' if and only if there exists a directed path from s to t in G .

For each node v_i in G , create two nodes v_{iL} and v_{iR} in G' . In addition, add nodes v_s and v_t to G' . Let the edges of G' be $E' = \{(v_{iL}, v_{iR}) : i \in [n]\} \cup \{(v_{iR}, v_{jL}) : (v_i, v_j) \in E\} \cup \{(v_s, v_{1L})\} \cup \{(v_t, v_{nR})\}$. Let the existing matching be $M = \{(v_{iL}, v_{iR}) : i \in [n]\}$. There is an augmenting path in G' if and only if there is a path from s to t in G . $\square$

4. Distances, girth and other problems

In this section, we consider the problems of computing shortest-path distances, diameter, and girth on graph streams. We briefly mention several other graph-stream problems at the end of this section.

First we show that, in the semi-streaming model, computing exact shortest-path distances, and even certain approximations, is not possible in one pass. In a graph G , we say an edge (u, v) is k -critical if the shortest path from u to v in $G \setminus (u, v)$ has length $\geq k$.

Lemma 5. *For $1 > \varepsilon > 0$ and sufficiently large n there exists a graph $G = (V, E)$ with $|V| = n$, $|E| = 2^{\log^\varepsilon n} n/4$ such that the majority of edges are $(\log^{1-\varepsilon} n)/2$ -critical and the majority of the subgraphs in the set*

$$\{G' : G' \text{ formed from } G \text{ by deleting a subset of the } (\log^{1-\varepsilon} n)/2 - \text{critical edges}\}$$

have diameter less than or equal to $4 \log^{1-\varepsilon} n$.

Proof. We show existence by considering a random graph $G \in \mathcal{G}_{n,p}$ where n is very large and $p = 2^{\log^\varepsilon n}/n$. Note that each edge appears in the graph independently with probability p . By Chernoff bound, with high probability, the number of edges in G is at least $(2^{\log^\varepsilon n}n)/4$.

Claim 1. *W.h.p the majority of edges are k -critical where $k = \log^{1-\varepsilon} n/2$.*

Proof of Claim 1. By Chernoff bound, for a vertex v , $\mathbb{P}(d(v) \geq 2 \cdot 2^{\log^\varepsilon n}) \leq (e/4)^{2^{\log^\varepsilon n}}$. By the union bound, the probability that any of the vertices in the graph has degree at least $2 \cdot 2^{\log^\varepsilon n}$ is at most $\sum_{v \in V} \mathbb{P}(d(v) \geq 2 \cdot 2^{\log^\varepsilon n}) \leq n \cdot (e/4)^{2^{\log^\varepsilon n}}$. Because for sufficiently large n , $2^{\log^\varepsilon n} \geq \log^2 n \geq 1$, it follows that $n \cdot (e/4)^{2^{\log^\varepsilon n}} \leq n \cdot (e/4)^{(\log_{4/e} n)^2} = 1/n^{\log n - 1}$. Hence, with probability $1 - 1/n^{\log n - 1}$ all the vertices have degree at most $2 \cdot 2^{\log^\varepsilon n}$. Henceforth, we assume that this is the case for all vertices.

Now, consider an edge (u, v) . Let $\Gamma_i(v)$ be the vertices up to a distance i from v in $G \setminus (u, v)$ and then

$$|\Gamma_k(v)| \leq \sum_{0 \leq i \leq k} (2 \cdot 2^{\log^\varepsilon n})^i \leq (2 \cdot 2^{\log^\varepsilon n})^{k+1} \leq 2^{2(\log n)/3}.$$

Observe that in the random graph, the vertex u is selected uniformly (with probability p) from the set of vertices $\{w | w \in V \setminus \{v\}\}$. The edge (u, v) is k -critical if u lies outside of $\Gamma_k(v)$. Hence the probability that (u, v) was k -critical is $1 - 2^{2(\log n)/3}/n = 1 - 1/n^{1/3} \rightarrow 1$. By Chernoff bound, with high probability, the majority of edges are k -critical. $\square$

Claim 2. *Consider $G \in \mathcal{G}_{n,p/2}$ w.h.p the diameter D of G is smaller than $4 \log^{1-\varepsilon} n$.*

Proof of Claim 2. Pick a node $v \in G$. We examine the i -neighborhood $\Gamma'_i(v) = \{w | w \text{ is distance } i \text{ away from } v\}$ while increasing the value of i . Let $S_i = \Gamma'_i(v) \setminus \Gamma'_{i-1}(v)$. For small i , $\Gamma'_i(v)$ contains few vertices. When an edge (x, y) , $x \in S_i$ and $y \in V \setminus \Gamma'_{i-2}(v)$, is selected into the random graph, y can be a vertex either in the set $S_i \cup S_{i-1}$ or in the set $V \setminus \Gamma'_i(v)$. In the latter case, y forms part of S_{i+1} . Particularly, for $|\Gamma'_i(v)| < n/2$, with probability larger than $\frac{1}{2} \cdot 2^{\log^\varepsilon n}/n$, an edge selected with one endpoint in S_i and the other endpoint in $V \setminus \Gamma'_{i-2}(v)$ may give rise to a vertex in the set S_{i+1} . Note that the edges are selected independently with the above probability from a set of $|S_i| \cdot (|V| - 1)$ edges. Hence, by the Chernoff bound, with high probability, for all i such that $|\Gamma'_i(v)| < n/2$ we have $|S_{i+1}| > |S_i| 2^{\log^\varepsilon n}/4$. We now increase the value of i and consider the first value t of i for which $|\Gamma'_t(v)| \geq n/2$. Given that $|\Gamma'_t(v)| = \sum_{j=1}^t |S_j|$ and for $i = 1, 2, \dots, t-1$, $|S_{i+1}| > |S_i| 2^{\log^\varepsilon n}/4$, we have $|S_t| \geq n/4$ and with high probability $|\Gamma'_{t+1}(v)| = n$. Hence the distance between v and every other vertex is $\leq \log n / (\log^\varepsilon n - 2) < 2 \log^{1-\varepsilon} n$ and so the diameter is $< 4 \log^{1-\varepsilon} n$.

Let $G \in \mathcal{G}_{n,p}$ and G' be a random subgraph of G . Picking a random subgraph of a graph picked from $\mathcal{G}_{n,p}$ is the same as picking a graph from $\mathcal{G}_{n,p/2}$. Consider the events

$$A = \{G \in \mathcal{G}_{n,p} : \text{majority of } E(G) \text{ are } k\text{-critical}\},$$

$$B = \{G \in \mathcal{G}_{n,p/2} : \text{diameter of } G \text{ is } < D\}$$

for each graph H ,

$$B_H = \{\text{subgraphs } H' \text{ of } H : \text{diameter of } H' \text{ is } < D\}.$$

From Claim 2 we know $\mathbb{P}(B)$ is large and from Claim 1 we know $\mathbb{P}(\neg A)$ is small. Thus, $\mathbb{P}(A \cap B) \geq \mathbb{P}(B) - \mathbb{P}(\neg A) > 1/2$. $\mathbb{P}(A \cap B) = \sum_G I[A] \mathbb{P}(G) \mathbb{P}(B_G)$ and so there exists a graph $G \in A$ such that $\mathbb{P}(B_G) \geq 1/2$, i.e. the majority of subgraphs of G have diameter $< D$. Now, undeleting edges that were not k -critical can only decrease the diameter and hence there exists at least one graph G such that the majority of $\{G' : G' \text{ formed from } G \text{ by deleting a subset of } k\text{-critical edges}\}$ have diameter $< D$. $\square$

Theorem 4. *For $1 > \varepsilon > 0$, it is impossible in one pass to approximate the diameter of an unweighted graph within a factor of $o(\log^{1-\varepsilon} n)$ in the semi-streaming model.*

Proof. Let $k = \log^{1-\varepsilon} n/2$ and $D = 8k$. Consider a graph G on $\eta = (n - 2D)/D$ vertices with the properties in Lemma 5. Let the subgraphs of this G with diameter $< D$ be $\mathcal{F}(G)$. Observe that

$$|\mathcal{F}(G)| \geq 2^{2^{\log^{\varepsilon} n}/8} = 2^{\omega(n \cdot \text{polylog} n)}$$

and hence, for any algorithm there exists two graphs $G', G'' \in \mathcal{F}(G)$ that are indistinguishable from information stored by the algorithm. Consequently, when we stream in one of G', G'' there exists an edge $e \in E(G') \setminus E(G'')$ whose existence in the streamed graph is undetermined. We stream in D graphs $G_1, \dots, G_D$ from $\mathcal{F}(G)$ such that there exists an edge (t_i, s_i) in each G_i whose existence is undetermined. Finally, we stream in edges (t_i, s_{i+1}) for $i = 2, \dots, D-1$ and two disjoint length D paths, one with endpoints s and t_1 and the other with endpoints s_D and t . Because of these two paths, the diameter is realized by the shortest path between s and t . Our construction gives a graph that has diameter $4D-1$ while the algorithm cannot guarantee that the diameter is less than $3D-1+Dk$. Hence the best approximation ratio is $\Omega(k)$. $\square$

We next show that the shortest-path distances can be approximated in one pass in the semi-streaming model. The approximation uses *graph spanners*. A subgraph $G'(V, E_s)$ is a t -spanner of graph $G(V, E)$ if, between any pair of vertices, the distance in G' is at most t times the distance in G . For an unweighted graph, a $\log n / \log \log n$ -spanner S can be constructed in one pass in the semi-streaming model using a simple algorithm similar to the one in [3]. Because a graph whose girth is larger than k can only have $\lceil n^{1+2/(k-1)} \rceil$ edges [5], the algorithm constructs S by adding the edges in the stream to S , if such an edge does not cause a cycle of length less than $\log n / \log \log n$ in the spanner S constructed so far. For a weighted graph, however, the construction in [3] requires sorting the edges according to their weights, which is difficult in the semi-streaming model. So, instead of sorting, we use a geometric grouping technique to extend the spanner construction for unweighted graphs to a construction for weighted graphs. This technique is similar to the one used in [7]. Let $\omega_{\min}$ be the minimum weight and let $\omega_{\max}$ be the maximum weight. We divide the range $[\omega_{\min}, \omega_{\max}]$ into intervals of the form $[(1+\varepsilon)^i \omega_{\min}, (1+\varepsilon)^{i+1} \omega_{\min})$ and round all the weights in the interval $[(1+\varepsilon)^i \omega_{\min}, (1+\varepsilon)^{i+1} \omega_{\min})$ down to $(1+\varepsilon)^i \omega_{\min}$. For each induced graph $G^i = (V, E^i)$, where E^i is the set of edges in E whose weight is in the interval $[(1+\varepsilon)^i \omega_{\min}, (1+\varepsilon)^{i+1} \omega_{\min})$, a spanner can be constructed in parallel using the above construction for unweighted graphs. The union of the spanners for all the G^i , $i \in \{0, 1, \dots, \log_{(1+\varepsilon)}(\omega_{\max}/\omega_{\min}) - 1\}$, forms a spanner for the graph G . Note that this can be done without prior knowledge of $\omega_{\min}$ and $\omega_{\max}$.

Theorem 5. For $\varepsilon > 0$, and a weighted undirected graph on n vertices, whose maximum edge weight, $\omega_{\max}$, and minimum edge weight, $\omega_{\min}$, satisfy $\log(\omega_{\max}/\omega_{\min}) = \text{polylog } n$, there is a semi-streaming algorithm that constructs a $(1+\varepsilon) \log n$ -spanner of the graph in one pass. The algorithm uses $O(\log_{(1+\varepsilon)}(\omega_{\max}/\omega_{\min}) \cdot n \log n)$ bits of space and the worst-case processing time for each edge is $O(\log_{(1+\varepsilon)}(\omega_{\max}/\omega_{\min}) \cdot n)$.

Once we have the spanner, the distance between any pair of vertices can be approximated by computing their distance in the spanner. The diameter of the graph can be approximated by the spanner diameter too. Note that, if the girth of an unweighted graph is larger than k , it can be determined exactly in a k -spanner of the graph. The construction of the $\log n / \log \log n$ -spanner thus provides a $\log n / \log \log n$ -approximation for the girth.

We end this section by briefly mentioning some graph problems that are simple in the semi-streaming model but may be impossible in a (poly)log-space streaming setting.

A minimum spanning tree can be constructed in one pass and $O(\log n)$ time per edge using a simple adaptation of an existing on-line algorithm [20]. Planarity testing is impossible in the (poly)log-space streaming model, because deciding the existence of a K_5 minor of a graph would require $\Omega(n)$ bits of space. Because a planar graph would have at most $3n-6$ edges, using $O(n)$ storage, many existing algorithms can be adapted to the semi-streaming model.

The following is an algorithm for finding articulation points in the semi-streaming model. It uses one disjoint set data structure, SF, for keeping track of the connected components of the spanning forest, T . It also uses one disjoint set data structure per vertex v , in order to store v 's neighbors.

Algorithm 5 (Articulation points).

$T = (V, \emptyset)$

For each $v \in V$: SF.makeset(v)

For each input edge (u, v) :
 if $SF.find-set(u) = SF.find-set(v)$ then:
 find the path, $u = a_0, a_1, \dots, a_k = v$ from u to v in T
 For each $a_i, 0 < i < k, a_i.union(a_{i-1}, a_{i+1})$.
 else:
 $SF.union(u, v)$
 $T = T \cup \{(u, v)\}$
 $u.makeset(v)$
 $v.makeset(u)$
 For each $v \in V$:
 if the neighbors of v w.r.t. T lie in at least two different sets
 then output v as an articulation point.

If u is an articulation point there exists two neighbors v and w of u in T such that any path from v to w passes through u . In this case, in the disjoint set structure for u the components containing v and w will never be unioned.

5. Conclusion

We considered a set of classical graph problems in the semi-streaming model. We showed that although exact answers to most of these problems are still impossible, certain approximations are possible. More research is needed for a complete understanding of the model. We propose the following open problems:

- (1) The efficiency of an algorithm in our semi-streaming model is measured by $S(m, n)$, $P(m, n)$ and $T(m, n)$ as in Definition 2. Together with the approximation factor, an algorithm in the semi-streaming model thus has four parameters. It would be interesting to develop a better understanding of the tradeoffs among these parameters.
- (2) Specifically, for spanners, is there a more time efficient one-pass algorithm? Is there a multipass algorithm that improves the approximation ratio?
- (3) Is there a better approximation to the maximum matching in general graphs with $O(1)$ passes?

References

- [1] J. Abello, A.L. Buchsbaum, J.R. Westbrook, A functional approach to external graph algorithms, *Algorithmica* 32 (3) (2002) 437–458.
- [2] N. Alon, Y. Matias, M. Szegedy, The space complexity of approximating the frequency moments, *J. Comput. Systems Sci.* 58 (1) (1999) 137–147.
- [3] I. Althöfer, G. Das, D. Dobkin, D. Joseph, Generating sparse spanners for weighted graphs, in: *Proc. 2nd Scandinavian Workshop on Algorithm Theory, Lecture Notes in Computer Science*, Vol. 447, Springer, Berlin, 1990, pp. 26–37.
- [4] Z. Bar-Yossef, R. Kumar, D. Sivakumar, Reductions in streaming algorithms, with an application to counting triangles in graphs, in: *Proc. 13th ACM–SIAM Symp. on Discrete Algorithms*, 2002, pp. 623–632.
- [5] B. Bollobás, *Extremal Graph Theory*, Academic Press, New York, 1978.
- [6] A.L. Buchsbaum, R. Giancarlo, J.R. Westbrook, On finding common neighborhoods in massive graphs, *Theoretical Comput. Sci.* 299 (1–3) (2003) 707–718.
- [7] E. Cohen, Fast algorithms for constructing t -spanners and paths with stretch t , *SIAM J. Comput.* 28 (1998) 210–236.
- [8] P. Drineas, R. Kannan, Pass efficient algorithms for approximating large matrices, in: *Proc. 14th ACM–SIAM Symp. on Discrete Algorithms*, 2003, pp. 223–232.
- [9] D. Eppstein, Z. Galil, G.F. Italiano, Dynamic graph algorithms, in: *Handbook of Algorithms and Theory of Computation*, CRC Press, Boca Raton, FL, 1999 (Chapter 8).
- [10] J. Feigenbaum, S. Kannan, M. Strauss, M. Viswanathan, An approximate L^1 difference algorithm for massive data streams, *SIAM J. Comput.* 32 (1) (2002) 131–151.
- [11] P. Flajolet, G.N. Martin, Probabilistic counting, in: *Proc. 24th IEEE Symp. on Foundation of Computer Science*, 1983, pp. 76–82.
- [12] A.C. Gilbert, S. Guha, P. Indyk, Y. Kotidis, S. Muthukrishnan, M. Strauss, Fast, small-space algorithms for approximate histogram maintenance, in: *Proc. 34th ACM Symp. on Theory of Computing*, 2002, pp. 389–398.
- [13] S. Guha, N. Koudas, K. Shim, Data-streams and histograms, in: *Proc. 33rd ACM Symp. on Theory of Computing*, 2001, pp. 471–475.
- [14] M.R. Henzinger, P. Raghavan, S. Rajagopalan, Computing on data streams, Technical Report 1998-001, DEC Systems Research Center, 1998.
- [15] P. Indyk, Stable distributions, pseudorandom generators, embeddings and data stream computation, in: *Proc. 41st IEEE Symp. on Foundations of Computer Science*, 2000, pp. 189–197.

- [16] B. Kalyanasundaram, G. Schnitger, The probabilistic communication complexity of set intersection, *SIAM J. Discrete Math.* 5 (1990) 545–557.
- [17] M. Karpinski, W. Rytter, *Fast Parallel Algorithms for Graph Matching Problems*, Oxford Lecture Series in Mathematics and its Applications, Oxford University Press, Oxford, 1998.
- [18] R. Kumar, P. Raghavan, S. Rajagopalan, A. Tomkins, Extracting large-scale knowledge bases from the web, in: *Proc. 25th Internat. Conf. on Very Large Data Bases*, 1999, pp. 639–650.
- [19] S. Muthukrishnan, *Data streams: algorithms and applications*, 2003, Available at <http://athos.rutgers.edu/~muthu/stream-1-1.ps>
- [20] R. Tarjan, *Data Structures and Network Algorithms*, SIAM, Philadelphia, PA, 1983.
- [21] R. Uehara, Z.-Z. Chen, Parallel approximation algorithms for maximum weighted matching in general graphs, *Inform. Process. Lett.* 76 (1–2) (2000) 13–17.